

[실습 1] Spring JPA 로 SpringBootLab6 N:M 관계를 구현

Spring Boot JPA(또는 Hibernate JPA)?

JPA 학습 단계

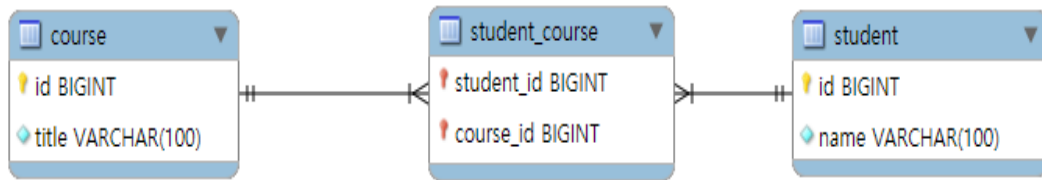
단계	내용
[1] 엔티티 선언	@Entity, @Id, @GeneratedValue 로 엔티티와 기본키 정의
[2] 관계 매핑	@ManyToOne, @OneToMany 등 관계 설정 (Fetch: LAZY, EAGER)
[3] 영속성 컨텍스트	1 차 캐시, Dirty Checking 으로 엔티티 상태 관리
[4] Repository	JpaRepository 상속 및 메서드명 기반 쿼리 자동 생성
[5] JPQL	@Query 로 복잡한 쿼리 작성
[6] 테스트	@DataJpaTest, @Transactional 로 Repository 단위 테스트

연관 관계 매핑

단방향/양방향 관계

@	설명	샘플 예제 (간단하게)
@ManyToOne	다대일 관계 (ex. 여러 사원이 한 부서에 소속)	@ManyToOne private Dept dept;
@OneToMany	일대다 관계 (ex. 한 부서에 여러 사원)	@OneToMany(mappedBy="dept") private List<Emp> emps;
@OneToOne	일대일 관계 (ex. 사원과 사원정보가 1:1)	@OneToOne private EmpInfo empInfo;
@ManyToMany	다대다 관계 (ex. 학생-강의)	@ManyToMany private List<Course> courses;

[실습] SpringLab06 Emp-Dept 구조 (1:N) JPA



1) 디렉토리 구조

```
SpringBootLab6 /
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/sec01/
│   │   │   │   ├── Sec01Application.java
│   │   │   │   ├── controller/
│   │   │   │   │   ├── StudentCourseController.java
│   │   │   │   │   ├── service/
│   │   │   │   │   │   ├── StudentCourseService.java
│   │   │   │   │   │   ├── repository/
│   │   │   │   │   │   │   ├── StudentRepository.java
│   │   │   │   │   │   │   ├── CourseRepository.java
│   │   │   │   │   │   │   ├── entity/
│   │   │   │   │   │   │   │   ├── Student.java
│   │   │   │   │   │   │   │   ├── Course.java
│   │   │   │   │   │   │   └── resources/
│   │   │   │   │   │   │       ├── application.yml
│   │   │   │   │   │   │       ├── templates/
│   │   │   │   │   │   │       │   ├── students.html
│   │   │   │   │   │   │       │   └── courses.html
```

2) Application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/spring_lab06
    username:
    password:
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update # 또는 create, create-drop, none
    show-sql: true # JPA 가 생성하는 SQL 을 콘솔에 출력
    properties:
      hibernate:
        format_sql: true # SQL 포매팅
  logging:
    level:
      org:
        hibernate:
          SQL: DEBUG # 실행되는 SQL 쿼리 로깅
        type:
          descriptor:
            sql: TRACE # SQL 파라미터 로깅
```

Entity

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}
```

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Course {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}
```

Repository

@Repository

public interface CourseRepository **extends** JpaRepository<Course, Long> {}

@Repository

public interface StudentRepository **extends** JpaRepository<Student, Long> {}

StudentCourseService

메서드명 : 반환타입	기능 설명	사용 Repository
getAllStudents() : List<Student>	모든 학생 목록을 조회한다.	StudentRepository
getAllCourses() : List<Course>	모든 과목 목록을 조회한다.	CourseRepository
addStudent(String, String) : void	학생과 과목을 동시에 추가하고, 중간테이블까지 함께 INSERT 한다.	StudentRepository (save) (JPA 가 중간테이블 insert 까지 처리)
addCourse(String) : void	과목만 단독으로 추가한다.	CourseRepository
deleteStudent(Long) : void	학생을 삭제한다.	StudentRepository
deleteCourse(Long) : void	과목을 삭제한다.	CourseRepository

StudentCourseController

메서드명 : 반환타입	HTTP 메서드	경로 (URL)	기능 설명
showStudents(Model) : String	GET	/students	모든 학생 목록과 과목 목록을 함께 조회한다.
showCourses(Model) : String	GET	/courses	모든 과목 목록을 조회한다.
addStudent(String, String) : String	POST	/students	학생과 과목을 동시에 추가한다.
addCourse(String) : String	POST	/courses	과목을 단독으로 추가한다.
deleteStudent(Long) : String	GET	/students/delete/{id}	특정 학생을 삭제한다.
deleteCourse(Long) : String	GET	/courses/delete/{id}	특정 과목을 삭제한다.

Template 구현 요구사항

1 메인 화면 (/students)

- 학생 목록을 테이블로 출력한다.
→ 학생의 **이름, 번호, 수강 중인 과목명**을 출력할 것.
- 학생 추가 폼을 구현한다.
→ 학생 이름, 과목명을 입력받아 POST /students 로 전송.

2. 목록

- 전체 과목 목록을 아래쪽에 테이블로 출력한다.
→ 과목의 **번호, 과목명**을 출력할 것.
- 과목 추가 폼을 구현한다.
→ 과목명만 입력받아 POST /courses 로 전송.

3. 삭제 능

- 각 학생/과목 옆에 **삭제 버튼**을 구현하고,
→ 클릭 시 각각의 /students/delete/{id}, /courses/delete/{id} 경로로 이동하여 삭제되도록 처리할 것.

[추가 실습 01] Controller 테스트를 구현한다

Controller

```
@GetMapping("/emp-info")
public String showEmpDeptInfo(Model model,
                               @RequestParam(defaultValue = "0") int page,
                               @RequestParam(defaultValue = "5") int size,
                               @RequestParam(defaultValue = "ename") String sortBy,
                               @RequestParam(defaultValue = "asc") String direction) {
    Sort sort = direction.equalsIgnoreCase("asc") ? Sort.by(sortBy).ascending()
                                                    : Sort.by(sortBy).descending();

    Pageable pageable = PageRequest.of(page, size, sort);
    Page<EmpDeptDto> empPage = service.getEmpDeptPage(pageable);
    model.addAttribute("empPage", empPage);
    return "emp-info";
}
```

View

```
<div>
    <a th:href="@{/emp-info?page=0&sortBy=ename&direction=asc}">이름
오름차순</a>
    <a th:href="@{/emp-info?page=0&sortBy=ename&direction=desc}">이름
내림차순</a>
    <a th:href="@{/emp-info?page=0&sortBy=sal&direction=asc}">급여
오름차순</a>
</div>
```